

CMSE 801 - Day 24

Markov Chain Monte Carlo Methods

Nov 30, 2022

General Course Announcements

- Homework #5 is due Wednesday Nov 30 at 11:59 p.m.
- Project's deadline Wednesday **Dec 7** at 11:59 p.m.
- Additional office hours today from 1:00-3:00 p.m. at room 2507H Engineering Building
- Bring your own computer and connector to present

```
In [1]: import numpy as np

names = ['Jamal', 'Lauren', 'Jiabao', 'Hasanur', 'Morgen', 'Arifuzzaman', 'Shuangshuan']

# np.random.choice(names, size=13, replace=False)
# volunteers to present Monday
# Monday = ['Raghav',]
# Wednesday = []
```

Questions/comments from Day 24 PCA

- Could you go over step 3 for loop part during the class?
- Is there some method for 3 or more parameters? / Could you talk more about part3 in class?
- Is there an easier way to build the chisquared_grid without using double indexing, example: Avalues[i][j], Bvalues[j][j]?
- Should we exhibit or run the code in the presentation?

$$\chi_R^2 = \frac{1}{N_{pts}} \sum_i \frac{(y_{data,i} - y_{model,i})^2}{2\sigma_i^2}$$

```
In [2]: %matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
```

```
In [3]: def my_model(A,B,sigma=0.5,data_points=10):
    """
    Creates a simple set of data points in a parabola that has
    noise added in the y-direction. The user can control the parabola
    properties (A and B) as well as the noise level and number of data
    points. Returns x and (noisy) y values.
    """
    x = np.linspace(-5,5,data_points)
    y = A*x**2 + B
    y_noisy = y + np.random.normal(0.0,sigma,data_points)
```

```
    return x, y, y_noisy
```

```
In [4]: def estimate_model_vals(A,B,x):  
        '''  
        Given x-values and our MCMC code's estimates for A and B, this returns  
        estimated y values that we can compare to the actual data .  
        '''  
        return A*x**2 + B
```

```
In [5]: A_user = 0.5  
        B_user = -3  
        sigma = 1.5  
        data_points=10  
        # now we're going to generate a model and create artificial "data" with some error  
        x_data, y_model, y_data = my_model(A_user,B_user,sigma,data_points)
```

```
In [6]: def chi_squared_error(ydata,ymodel,error):  
        '''  
        Takes in the observed data, our model's estimated y-values  
        for the model, and our estimate for the errors in the data.  
  
        Returns sum-of-squares error (chi-squared error)  
        '''  
        return ((ydata-ymodel)**2/(2*error**2)).sum()/(ydata.size)
```

```
In [7]: Astart = -4  
        Aend = 4  
        deltaA = 0.1  
        Bstart = -10  
        Bend = 8  
        deltaB = 0.2  
        Avalues, Bvalues = np.mgrid[Astart:Aend:deltaA, Bstart:Bend:deltaB]  
        chisquared_grid = np.zeros_like(Avalues)
```

```
In [8]: for i in range(chisquared_grid.shape[0]):  
        for j in range(chisquared_grid.shape[1]):  
            this_model = estimate_model_vals(Avalues[i,j],Bvalues[i,j],x_data)  
            chisquared_grid[i,j] = chi_squared_error(this_model, y_data, sigma)  
        print("min, max error values are:", chisquared_grid.min(),chisquared_grid.max())  
  
min, max error values are: 0.12043124972220416 991.2143708520938
```